

AI Log Analyzer — Cowork session export

Session date: July 16–17, 2026 · User: TG · Assistant: Claude (Cowork) · Project folder: D:\AI Log Analyzer

Part 1 — Summary

Goal

Build a production-quality, portfolio-worthy AI Log Analyzer: a platform that ingests application/infrastructure logs, groups and classifies errors, identifies root causes with confidence scores, explains failures in plain language, and suggests (never executes) remediation commands for Linux, Kubernetes, and AWS. Development is run in mentor mode: every module is explained (business problem, architecture, tradeoffs, line-level detail, interview questions) before moving on.

Key decisions

Decision	Choice	Rationale (short)
LLM provider	OpenAI, behind a provider interface	Mockable in tests, swappable later
Architecture	Modular monolith (FastAPI)	Microservices solve team-scale problems we don't have; module seams allow later split
AI jobs	Celery + Redis, 202 Accepted + polling	AI calls take 10-60s; can't hold HTTP requests open
Cost/quality lever	AI analyzes grouped errors, not raw lines	10,000 duplicate lines become one analysis
Vector store	Qdrant (not FAISS)	Real ops story: persistence, filtering, runs in compose/K8s
LangChain	Not used for core	Thin, debuggable code preferred
Safety boundary	Suggest commands, never execute	Deliberate product + security decision

Work completed

Module 0 — Planning (complete). Requirements & scope (personas, user stories, FR/NFR, risks), architecture doc with Mermaid diagrams and tradeoff analysis, roadmap mapping 10 AI phases onto milestones M0-M10, README, CONTRIBUTING with git strategy (GitHub Flow, conventional commits, ADRs).

Module 1 — Backend skeleton (complete, verified). FastAPI app factory with lifespan; typed settings via pydantic-settings (fail-fast validation, cached singleton); structured JSON logging via structlog; liveness + readiness health endpoints with Pydantic response schemas; dependency-injection wiring (SettingsDep); pytest suite using in-process httpx ASGITransport — 7/7 tests passing; multi-stage Dockerfile (non-root user, layer-cache optimization); docker-compose with postgres 16 + redis 7 and healthchecks; GitHub Actions CI (ruff, black, mypy, pytest); pre-commit hooks; full explainer at docs/modules/module-01-backend-skeleton.md.

Side items. evlog agent skills: the requested npx skills add <https://www.evlog.dev> found no manifest from the sandbox; installing from the GitHub repo (hugorcd/evlog) yielded three contributor skills into .claude/skills/ — not the user-facing log-analysis skills, which require running the command on the user's own machine. A UI mockup of the analysis dashboard was rendered in-chat (error groups sidebar, root cause card, suggested commands, confidence score).

Roadmap position and next steps

Position: Milestone 1 of 10 complete. Pending on the user's machine: git init + first commits (git can't run through the sandbox mount), copy backend/.env.example to backend/.env, then docker compose up --build and verify <http://localhost:8000/docs>. Next: Module 2 — Auth & users (SQLAlchemy async + Alembic, JWT access/refresh, rate limiting, audit log, repository pattern).

Part 2 — Full transcript

User messages are shown verbatim on shaded background. Assistant replies are the user-visible text; bracketed lines summarize file/tool activity. The initial master prompt is abridged to its section headers for readability.

Turn 1 — user (master prompt, abridged)

Act as a Senior Software Engineer, AI Engineer, DevOps Engineer, and Technical Mentor with 15+ years of experience. I want to build a production-quality AI Log Analyzer from scratch that is resume-worthy... [Sections: Objective / Development Style (mentor mode, one module at a time) / Tech Stack (Python 3.12, FastAPI, SQLAlchemy, PostgreSQL, Redis, Celery, OpenAI, FAISS/Qdrant, Docker, K8s, GitHub Actions, React+TS+Tailwind, Prometheus/Grafana, JWT, pytest, ruff/black/mypy) / Functional Requirements / AI Features in 10 phases / Example input+output / SE Requirements (Clean Architecture, SOLID, Repository Pattern, DI) / Folder Structure / Git Strategy / Docker / Kubernetes / Database Design / API Design / Testing / Security / Performance / Documentation / Learning Mode / Development Rules: start with planning, requirements, architecture, milestones, repo setup before code.] Trailing line: npx skills add <https://www.evlog.dev>

Turn 1 — assistant

Asked four setup questions (AskUserQuestion): LLM provider -> OpenAI API; workspace -> folder on user's computer (connected D:\AI Log Analyzer); mentoring depth -> full mentor mode; experience -> solid Python, new to FastAPI. Then delivered Module 0: [Created docs/planning/01-requirements.md, 02-architecture.md, 03-roadmap.md, README.md, CONTRIBUTING.md, .gitignore]. Explained the three key decisions: modular monolith over microservices; async analysis via Celery with 202 Accepted; AI analyzes grouped errors rather than raw lines. Git init failed through the sandbox mount (lock-file operations unsupported), so provided commands for the user to run locally: git init -b main; git add -A; git commit. Flagged that the npx skills line was not executed without confirmation, and noted the suggest-don't-execute safety boundary. Announced Module 1 as next.

Turn 2 — user

npx skills add <https://www.evlog.dev>

Turn 2 — assistant

Fetches evlog.dev: a TypeScript logger ('wide events', structured errors with why/fix fields) that ships agent skills. Ran the command in the project; the website manifest was unreachable from the sandbox, so installed from the GitHub repo instead. [Installed .claude/skills/: create-evlog-adapter, create-evlog-enricher, create-evlog-framework-integration] Explained these are contributor skills for developing evlog itself — not the documented user-facing skills (review-logging-patterns, analyze-logs, build-audit-logs), which the user can get by running the command locally. Assessed relevance: evlog is TS (backend is Python) but its why/fix error schema is a good model for the analyzer's output format (M4), a candidate frontend logger (M6), and its NDJSON format a good parser test fixture (M3). Offered to delete the three skills.

Turn 3 — user

now start building project from scratch and keep every module of project segregated to understand it easily.

Turn 3 — assistant

Built Module 1 (backend skeleton), 21 files: [backend/pyproject.toml (deps + ruff/black/mypy/pytest config), .env.example, .pre-commit-config.yaml, app/main.py (app factory + lifespan), app/core/config.py (pydantic-settings), app/core/logging.py (structlog), app/api/deps.py (DI), app/api/v1/health.py (liveness + readiness), app/schemas/health.py, tests (conftest + 7 tests), infra/docker/api.Dockerfile (multi-stage, non-root), docker-compose.yml (api + postgres + redis), .github/workflows/ci.yml, docs/modules/module-01-backend-skeleton.md] Verified in sandbox: 7/7 tests passing (sandbox is Python 3.10; CI enforces 3.12 + lint). Explained the three concepts to internalize: app factory + DI over globals; liveness vs readiness probes; fail-fast config validation. Gave the user local commands (feature branch, commit, docker compose up --build) and announced Module 2 (auth and users) as next.

Turn 4 — user

is it done ?

Turn 4 — assistant

Clarified: Module 1 is done and verified (7/7 tests); the project overall is at Milestone 1 of 10. Listed the two steps pending on the user's machine: git init/commit, and copying .env then docker compose up --build, verifying /api/v1/health and /docs.

Turn 5 — user

on a side note can you tell me how user interface will look like while dependencies are getting installed can you geerate image for it

Turn 5 — assistant

[Rendered an interactive dashboard mockup in-chat] Layout: header with file metadata and upload/history actions; left rail of deduplicated error groups with severity badges (Critical x212 DB connection timeout, High x38 Redis pool exhausted, Medium x9 slow query, Low x3 deprecation warning); main panel with root cause, suggested fix, recommended kubectl/nc commands, 94% confidence; action row mapping to later milestones — chat with log (M7), incident report (M8), similar incidents (M5). React build lands in Milestone 6.

Turn 6 — user

i want to share this chat please help me with it

Turn 6 — assistant

Searched Anthropic support docs: regular chats have a Share button creating a public link, but Cowork sessions cannot be shared that way. Offered an export instead; user chose summary + full transcript as PDF. [Created this document: docs/session-export-2026-07-17.pdf]

Repository state at export

```
D:\AI Log Analyzer
+-- README.md / CONTRIBUTING.md / .gitignore / .pre-commit-config.yaml
+-- docker-compose.yml
+-- .github/workflows/ci.yml
+-- .claude/skills/          (3 evlog contributor skills)
+-- docs/
|   +-- planning/ 01-requirements.md 02-architecture.md 03-roadmap.md
|   +-- modules/  module-01-backend-skeleton.md
|   +-- session-export-2026-07-17.pdf (this file)
+-- infra/docker/api.Dockerfile
+-- backend/
|   +-- pyproject.toml .env.example
|   +-- app/  main.py | core/ config.py logging.py | api/ deps.py v1/ health.py router.py
|   |      schemas/ health.py
|   +-- tests/ conftest.py | unit/ test_health.py test_config.py      [7/7 passing]
```